第2回 演習・宿題

LoRA で回答形式を学習し、学習前後を比較する

松尾・岩澤研究室
Matsuo-Iwasawa Lab UTokyo

2026/10/26 演習：佐藤良明

小さなモデルを LoRA で短時間学習し、何が変わり、何が変わらないかを数字と出力で確かめる

許諾なく撮影や第三者への開示を禁止します

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

第1回 宿題の解説

素の LLM に質問して見えたことを整理し、今日の演習につなげる（最初の 5〜10 分）。

基礎	実装	発展
同じ質問を 3 回ずつ、モデルや temperature を変えて実行し、回答の違いをまとめる	app/ask.py を改修し、会話履歴を保ったまま続けて質問できるようにする	応答時間とトークン数を計測し、複数のモデルや質問で比べる。無料枠のレート制限との関係も考える

見るのは「何が起きたか」と「なぜそうなるか」。正解の数字を当てる課題ではない

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

基礎の解説：同じ質問でも答えは変わる

生成は確率的。同じ条件でも答えが変わる。第2回の講義の「サンプリング」につながる。

見えること

- 同じ条件で 3 回聞いても、実行ごとに答えが変わる
- モデルを変える（AIE_MODEL）と、文面・長さ・知らないことへの答え方が変わる

比べる観点

- 文面
- 長さ
- 事実の正誤
- 確信の強さ
- 講座に固有の質問への答え方

temperature：Google が 2026-07-21 に非推奨にした。Gemini 3.6 以降のモデルは指定しても無視する。効かない・エラーになったら、その結果をそのまま記録すればよい

gemini-3.5-flash（lite なし）：非推奨で、要求は gemini-3.6-flash へ転送される。比べるなら gemini-3.5-flash-lite と gemini-3.8-flash

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

実装の解説：会話履歴を保つとは

モデルは前のやり取りを覚えていない。毎回、これまでのやり取りを入力に含めて送る。

方法 1：SDK の Chat を使う

client.chats.create で Chat を作り、send_message で続けて送る

履歴は Chat が持ち、毎回まとめて送ってくれる

方法 2：履歴を自分で持つ

ユーザーとモデルの発言をリストにためる

毎回そのリストを generate_content の contents に渡す

解答例：方法 1 で app/ask.py を直す。次の 3 枚で、修正前・修正後・送られている中身を見る。

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

実装の解説：修正前の app/ask.py

ask() は質問を 1 つ受け取り、generate_content を 1 回呼んで答えを返すだけ。

修正前（抜粋）

```
def ask(question: str) -> str:
    client = genai.Client(api_key=get_api_key())
    response = client.models.generate_content(model=MODEL, contents=question)
    # TODO(第1回 演習): response.usage_metadata から
    #    入力トークン数と出力トークン数を表示してみよう（Copilotに頼んでよい）
    return response.text
```

main は argparse で question（省略時は DEFAULT_QUESTION）を受け、ask() の結果を表示する。errors.APIError は api_error_message で表示して終了する。

足りないこと 1：呼ぶたびに新しい generate_content を 1 回だけ呼ぶ

足りないこと 2：前の質問と答えを入力に含めないので、続けて聞いても覚えていない

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

実装の解説：修正後（解答例）の　chat()

chat() を足して、Chat に履歴を持たせたまま続けて質問できるようにする。

修正後（解答例、追加した関数）

```
def chat(first_question: str) -> None:
    client = genai.Client(api_key=get_api_key())
    session = client.chats.create(model=MODEL)  # 履歴は SDK の Chat が持つ
    question = first_question
    while question:
        response = session.send_message(question)  # これまでの履歴＋今回の質問を送る
        print(response.text)
        usage = response.usage_metadata
        print(f"[tokens] 入力 {usage.prompt_token_count} / 出力 {usage.candidates_token_count}")
        question = input("\n[続けて質問（空行で終了）] ").strip()
```

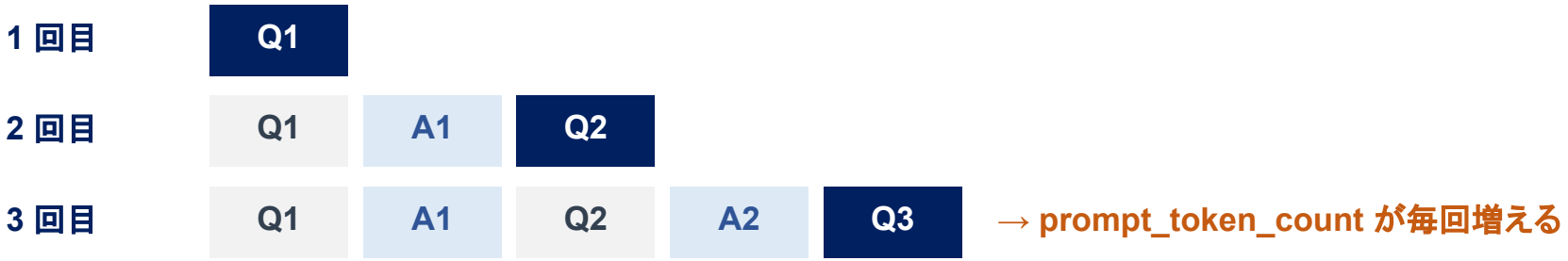
1. client.chats.create で Chat を作る（履歴を持つ）
2. send_message をループで呼ぶ（空行で終了）
3. --chat オプションを足し、ask() は残す（既存の使い方とテストを壊さない）

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

実装の解説：毎回、何が送られているか

送る内容＝これまでの履歴＋今回の質問。ターンを重ねるほど増えていく。

```
uv run python -m app.ask --chat "LoRAとは何ですか？"  # 空行で終了
```



紺：今回の質問　灰：これまでの質問　薄青：これまでの答え

入力トークンが増えると、コストと上限に効く。古い履歴の切り捨てや要約が必要になる

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

発展の解説：応答時間・トークン数・レート制限

1 回の値で決めず、複数回測って比べる。トークン数は応答に含まれる。

応答時間

- time.perf_counter() で呼び出しの前後を測る
- 1 回目は遅いことがあるので複数回測る
- 中央値で比べる

トークン数（ usage_metadata）

- 入力 prompt_token_count
- 出力 candidates_token_count
- 思考 thoughts_token_count
- 合計 total_token_count

思考の分は別に数えるので、入力＋出力が合計に合わないことがある

レート制限

- モデルごと・プロジェクトごとに、毎分・1 日のリクエスト数などの上限
- 超えると 429 エラー（例：429 RESOURCE_EXHAUSTED）。待って再試行する
- 現在の上限は AI Studio で確認

講座での実測（参考）：gemini-3.5-flash-lite は毎分 15・1 日 500、gemini-3.8-flash は毎分 5・1 日 10〜20 前後。上限は変わる。

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

つまずきやすい点と注意

第1回の手順と宿題でつまずきやすい点を、もう一度確認する。

1	**API キー**	コード、answer.md、ログ、スクリーンショットに書かない。漏れたらAI Studio で削除して作り直す
2	**PR の向き**	base repository が講座 repo になっていないか。自分の fork の main に向ける
3	**Codespaces**	使い終わったら停止する。起動したままだと無料枠を消費し続ける
4	**レート制限**	上限に当たったら、待つか回数を減らす。減らしたことはanswer.md に書く

今日の Colab でも同じ。使い終わったらランタイムを削除する

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

今日の演習へのつながり

第1回はモデルに「質問した」。今日は小さなモデルを自分で「学習させる」。

第1回

API の向こうのモデルに質問した

知らないことでも、確信を持って答えることがあった

→

今日（第2回）

小さなモデルを LoRA で学習させる

形式は学習で変わるが、知らないことへの判断は保証されない

第1回で見た「知らないことを答える」を、今日は学習の前後で確かめる

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

この回のゴール

学習で変わることと変わらないことを、自分の結果で説明できるようにする。

1

小さなモデルを LoRA で短時間学習させ、学習の前後で**何が変わり、何が変わらないか**を数字と出力で確かめる

2

「形式は学習で身につくが、知らないことへの判断（確認を求めるか、断定するか）までは保証されない」ことを、自分の結果で説明できる

3

ファインチューニングが向く場面と向かない場面を、自分の言葉で説明できる

演習中に使うもの： Google Colab（T4 GPU、無料枠）

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

何を確かめるか

形式は学習で整う。判断まで整うとは限らない。それを順に確かめる。

1	プロンプトで形式を指示しても、小さなモデルは守れないことがある	**セクション 3、学習前**
2	LoRA で数分学習すると、形式はほぼ守れるようになる	**セクション 5、学習後**
3	形式が整っても、FAQ に無いことを聞かれたときの判断（確認を求めるか、断定してしまうか）までは保証されない	**セクション 5、6**
4	**発展**：同じモデルを 4bit 量子化で読み込むと、学習しなくても出力が変わる	**セクション 8（任意）**

セクション：ノートブック notebooks/lora_format_tuning.ipynb の見出し番号。セルは上から順に実行する。

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

題材：架空の講座　FAQ と望ましい出力

FAQ と質問を入力し、JSON 1 行で答えさせる。FAQ の内容はすべて架空。

架空 FAQ（5 項目）

項目	内容（架空）
提出期限	9 月 30 日 23:59
提出先	学習ポータルの「課題」欄
提出形式	PDF 1 ファイル、学籍番号_課題番号.pdf
オフィスアワー	毎週火曜 17:00〜18:00
遅延提出	48 時間以内なら減点のうえ受け付け

望ましい出力

```
{"answer": <文字列>, "needs_review": <真偽値>}
```

根拠あり（FAQ にある）

FAQ の文で答え、needs_review は false

根拠なし（FAQ に無い）

「この FAQ には記載がありません。運営に確認が必要です。」

needs_review は true

架空データ：データはノートブックが生成する。実在の締切やURL は含まない。

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

データの分け方：学習で見た文を評価に出さない

言い換えをテンプレート単位で train／valid／test に振り分ける。

train

14 件

根拠あり 10、根拠なし 4。学習に使う

valid

7 件

train・test とは別のテンプレート

test

21 問

根拠あり 15、根拠なし 6。学習前後の評価に使う

正解は gold に分ける：needs_review と必須語句の正解はモデルに渡さない

学習量：データを複製せず、エポック数で決める

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

評価の 2 本立て：形式と内容を分ける

JSON の形式を守れたかと、答えが正しいかを、別々の指標で見る。

形式遵守率（format_rate）

- JSON として読める
- キーがちょうど answer と needs_review の 2 つ
- 型が文字列と真偽値

内容正解率（content_rate）

- needs_review が正解と一致する
- 正解の語句が answer に含まれる

キーが欠けると、文面が正しくても内容は　×

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

事前準備（任意、5 分）と使うもの

授業の前にセクション 0〜3 を一度実行しておくと、当日の詰まりが減る。

事前準備（任意）で確かめること

- GPU が割り当てられる
- ライブラリを導入できる
- モデルをダウンロードできる

Colab はセッションが切れると変数が消える。当日はあらためて 0 から実行する

使うもの

- Google Colab（T4 GPU、無料枠）
- モデル Qwen/Qwen3.5-0.8B（Apache-2.0）
- ノートブック notebooks/lora_format_tuning.ipynb

Colab で開く

https://colab.research.google.com/github/matsuolab/lecture-ai-engineering-2026/blob/main/notebooks/lora_format_tuning.ipynb

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

演習 45 分の流れ（最初に第 1回の宿題の解説）

最初の 5〜10 分で第1回の宿題を解説し、残りで手順 1〜7 を進める。

手順	やること	ノートブック・目安
—	第1回 宿題の解説	最初の 5〜10 分
1	Colab でノートブックを開き、自分用にコピーする	
2	ランタイムのタイプをT4 GPU にする	
3	上から実行し、学習前の表を読む	セクション 0〜3、約 5 分
4	LoRA で学習する。待ち時間にLoRA と QLoRA の違いを読む	セクション 4、4〜5 分
5	学習後を評価し、学習前と比べる。知識を問う質問を見る	セクション 5〜6
6	result.json を作り、answer.md を書いて PR を出す	セクション 7
7	ランタイムを接続解除して削除する	

T4 での目安: セクション 0〜3 で約 5 分、セクション 4〜7 で約 10 分（学習 4〜5 分を含む）。

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

手順 1〜2　自分用にコピーし、T4 GPU を選ぶ

自分のドライブにコピーしてから実行する。GPU は T4 を選ぶ。

1 Colab でノートブックを開き　ファイル → ドライブにコピーを保存

2 コピーしたノートブックで　ランタイム → ランタイムのタイプを変更 → T4 GPU

「GPU が見つかりません」と出たとき

ランタイム → ランタイムのタイプを変更 → T4 GPU を選び直す。混雑しているときは時間をおく。割り当てられないときは「学習できなかったとき」のスライドの方法で進める

セルは上から順に実行する。飛ばすと「変数が見つからない」エラーになる

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

手順 3　セクション 0〜3：学習前を評価する（約　5 分）

学習前の表で、どのキーが欠けているか、文面は FAQ と合っているかを読む。

見るところ

- セクション 0：GPU が Tesla T4 と表示される
- セクション 3：量子化なし（32bit）で読み込む。重みのメモリの表示例3.01GB
- 要約（format_rate、content_rate など）と 21 問の表
- 学習前は、多くの出力でneeds_review が欠けているはず

出ても問題ないもの

pip の赤い ERROR: pip's dependency resolver ...（cudf、pyarrow）：無視してよい

You are sending unauthenticated requests to the HF Hub：公開モデルはログイン不要

... is falling back to its reference PyTorch implementation ...：線形注意の高速実装が無いので遅いが、正しい実装で動く

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

手順 4　セクション 4：LoRA で学習する（4〜5 分）

20 エポック＝学習データ 14 件を 20 周、1 周 2 ステップで合計 40 ステップ。

見るところ	目安
trainable params ... trainable%	学習するのは全体の数 % だけ
Training Loss（10 ステップごと）	1.5 前後から、40 ステップで 0.01〜0.02 まで下がる
学習時間: ... 秒 | ステップ数: 40 | エポック: 20.0	T4 で 240〜290 秒

設定（CONFIG）の要点

lr 2e-4、lora_r 16、lora_alpha 32、seed 42、warmup_steps 6。生成は do_sample: False（貪欲生成で前後の条件を揃える）

1 ステップの中身

バッチ 4 × 勾配累積 2 ＝ 1 ステップ 8 件。cosine 減衰と warmup

待ち時間に：次のスライドの「LoRA と QLoRA の違い」を読む。

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

LoRA と QLoRA の違い

どちらも基盤モデルの重みは固定し、追加した小さな行列（adapter）だけを学習する。

観点	LoRA（今回）	QLoRA（基盤モデルを 4bit で保持）
重みのメモリ	大	小（16bit の約 1/4。70 億パラメータで 16bit 約 14GB、4bit 約 3.5GB）
基盤モデルの出力	元のまま	量子化誤差で変わることがある
学習の速さ	基準	速くはならない。遅くなることが多い
依存ライブラリ	peft	peft＋bitsandbytes（CUDA 前提）

QLoRA でも adapter は量子化しない。前後比較は同じ量子化条件で行い、adapter は基盤モデルの ID・revision・量子化設定と一緒に記録する。

モデルが GPU に載るなら LoRA、載らないなら QLoRA か、より小さいモデル

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

手順 5　セクション 5：学習後を評価し、学習前と比べる

学習前と同じ 21 問・同じ生成設定。変えたのは adapter の有無だけ。

比較表

format_rate と content_rate は、学習の前後でどう変わったか

退行

学習前は通っていたのに、学習後に落ちた質問。空が期待値

残った失敗の型

- 根拠なしの質問に FAQ の別項目を当てて**断定**
- 根拠ありの質問に「記載がありません」と**逃げた**

学習後の回答：教師データの文（FAQ 原文）をほぼそのまま返す形になる。形式と文面は揃うが、「FAQ に根拠があるか」の判断まで学習できたとは限らない。

seed を固定しても、GPU 上の学習は完全には再現しない。隣の人と残った質問が違っても異常ではない。失敗の型で比べる

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

手順 5　セクション 6：知識を問う質問

FAQ にも学習データにも無いことを聞いたとき、確認を求めたか、断定したか。

質問

FAQ に無い事実 3 問（望ましい応答は「確認が必要」）

- 最終課題の締切
- 担当教員の名前
- 次回の講義のテーマ

対照 1 問：課題の提出期限

分類	意味
format_error	形式を守れていない（学習前に多い）
needs_review	確認が必要と答えた（望ましい）
asserted	needs_review false で言い切った。FAQ に無い質問なら根拠のない断定

出やすい結果：学習前は形式違反になりやすく、学習後は確認が必要と断定の両方が出ることがある。

「Fine-Tuning では知識は絶対に直らない」と先に結論にせず、今回のデータでの結果を報告する

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

手順 6〜7　result.json を作って提出し、ランタイムを削除する

セクション 7 で result.json を作り、要点を lectures/02/answer.md に書いて PR を出す。

answer.md の「演習」に書くこと

- 使ったモデル・設定（result.json の config から）
- 自分で学習を実行したか（executed_training_myself）
- 学習前と学習後の format_rate・content_rate
- 退行した質問
- 学習後に残った失敗と、その型
- 知識を問う質問の結果（学習前の分類 → 学習後の分類）
- 学習で変わったこと・変わらなかったこと

1. result.json、eval_before.json、eval_after.json は /content/qlora_lab。左のファイル一覧で右クリック → ダウンロード
2. 要点を lectures/02/answer.md に書く。GitHub の画面上で編集してもよい
3. 作業ブランチ lec02 から自分の fork の main へ PR。result.json は lectures/02/ に置いて含めてもよい

手順 7：ランタイム → ランタイムを接続解除して削除。 Colab の GPU は無料枠で使える時間に限りがある

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

学習できなかったとき（ノートブックの　4b）

途中で止まったら再開する。学習できなければ、講師の結果を見て比較と考察を進める。

途中で止まった

チェックポイント（10 ステップごと、ckpt/）から再開する

```
trainer.train(resume_from_checkpoint=True)
```

GPU が割り当てられない・時間内に終わらない

講師が T4 で実行した result.json、eval_before.json、eval_after.json を見て、比較と考察を進める

置き場所：講座 repo の lectures/02/reference/。GitHub の画面で開くか「Download raw file」

提出時は executed_training_myself を False にし、自分で実行できた範囲を　answer.md に書く

CONFIG の USE_DRIVE を True にすると、adapter と結果を Google Drive にも保存でき、セッションが切れても残る

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

発展：4bit 量子化で読み込む（セクション　8、任意、約 2 分）

同じモデルを 4bit（NF4）で読み込み、学習はせずに、メモリと出力の変化を見る。

重みのメモリ

32bit → 4bit

約 3GB → 1GB 未満

出力の変化は方向が一定でない

- 良くなる質問もある
- needs_review が "false"（文字列）になり、型検査に落ちる
- 根拠なしの質問に「はい」「なし」と断定する

読み方：32bit の学習前との差は、量子化の影響。bitsandbytes は CUDA 前提で導入 1 分ほど、計算精度は float16（T4 は bf16 非対応）。

QLoRA で試すとき：prepare_model_for_kbit_training を通してから、セクション4 と同じ手順で学習する。学習前後とも4bit の基盤モデルで評価し、量子化の影響と学習の影響を混ぜない。

4bit のまま学習する　QLoRA も、T4 で動作を確認済み。宿題で試してよい

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

宿題：どれか 1 つで完了

基礎・実装・発展の 3 つのレベルから、1 つを選んで取り組む。

レベル	内容
基礎	CONFIG の lora_r・lr・num_train_epochs のどれか 1 つを変えて再学習し、format_rate・content_rate・最終 loss・学習時間を元の設定と並べて記録する。変えたら ランタイム → セッションを再起動 して上から実行する（学習後のモデルが残っているため）。result.json を 2 つ比べる
実装	FAQ・GROUNDED・UNGROUNDED を自分の題材（5 項目程度）に置き換えて学習し、学習前後を比べる。JSON の形式と needs_review の設計はそのまま使う。「根拠なし」の質問をどう用意したかも書く
発展	scripts/make_sft_dataset.py で講座の用語集から Q&A データ（messages 形式の JSONL）を作り、FAQ の代わりに学習させる。評価用の質問（用語集にある語・言い回しを変えた質問・用語集にない語）と正解を自分で用意し、学習前後を比べる。用語集を検索する仕組みと比較して、モデルを学習させる際のメリットとデメリットについて考察する

進め方と注意： 次のスライド。提出は lectures/02/answer.md の「宿題」欄。

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

宿題の進め方と注意

設定やデータを変えたら、条件をそろえて上から実行し直す。

レベル	進め方
基礎	設定を変えたら ランタイム → セッションを再起動 して上から実行する。result.json を 2 つ並べる
実装	「根拠なし」の質問を自分で用意し、どう用意したかを書く
発展	Codespaces で uv run python scripts/make_sft_dataset.py > glossary_sft.jsonl を実行し、できたファイルを Colab にアップロードする。各行は to_chat と同じ messages 形式なので、Dataset.from_list にそのまま渡してセクション 4 の SFTTrainer で学習できる。評価（test と gold）は用語集に合わせて自分で作り直す

注意 1：学習後は変数 model にも adapter が差し込まれている。学習前の出力を取り直すときは tuned.disable_adapter()（セクション 6）を使うか、セッションを再起動する

注意 2：ライブラリは動作確認した版（セクション 0 の PINS）に固定している。版を変えると動かなくなることがある

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

宿題の提出

演習と同じ lectures/02/answer.md の「宿題」欄に書いて提出する。

answer.md の「宿題」に書くこと

- 取り組んだレベル（基礎／実装／発展）
- 変えた設定やデータ
- 結果（比較表や数字）
- ファインチューニングが向く場面・向かない場面についての考察

締切：原則として講義日から1 週間

完了：どれか 1 つのレベルで完了

AI ツール：使ってよい。自分で確かめたことを書く

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

第3回の予告

第3回はエージェントを作る回。

今回

モデルに学習させて、答え方を変える

→

第3回

「用語集を検索するエージェント」を作る

第3回の内容と準備の詳細は、第 3回の資料で案内する

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO

まとめ：今日の到達点と次回

学習で変わったことと、変わらなかったことを、自分の結果で確かめた。

1

形式は、数分の LoRA 学習で変わる

2

知らないことへの判断（確認を求めるか、断定するか）は保証されない

3

ファインチューニングが向く場面と向かない場面を、自分の結果で説明する

宿題：基礎・実装・発展のどれか1つ。締切は原則として講義日から1 週間

第3回：エージェントを作る

Colab は使い終わったらランタイムを削除する

©MATSUO-IWASAWA LAB, THE UNIVERSITY OF TOKYO